SIMATS ENGINEERING

Department of Computer Science & Engineering

ASSESSMENT TOOL 1- ANALYTICAL PROBLEM SOLVING

Course Code:	**CSA1205**	**Course Title:**	**Computer Architecture**
CO Assessed:	**CO2**	**Assessment:**	**ASSESSMENT TOOL1-ANALYTICAL PROBLEM SOLVING**
Weightage:	**45%**	**Total Marks:**	**25**
CO2 Statement:	Apply integer and floating-point arithmetic algorithms including Booth's multiplication, restoring/non-restoring division, and IEEE 754 standard operations to solve fixed-point and floating-point computational problems. (BL3)		
Student Name:	**Yuvasri.s**	**Reg.No.:**	**192521032**
Department:	**B.TECH IT**	**Date:**	**25/07/2026**

ANNEXURE A

1. A processor performs arithmetic operations on signed 8-bit integers using 2's complement representation. Consider two numbers: +45 and −23. Analyze in detail how the system performs both addition and subtraction, including binary conversion, alignment of sign bits, and intermediate steps. Determine whether overflow occurs, and explain how the processor detects overflow and interprets the final result.

Given:

- Processor uses **8-bit signed integers** in **2's complement representation**.
- Number 1 = **+45**
- Number 2 = **−23**

The processor performs arithmetic by converting both numbers into 8-bit binary, aligning the sign bits, executing the operation, checking for overflow, and interpreting the final result.

1. Binary Conversion

(a) Convert +45

45_{10} = **00101101_2**

Since it is a positive number, its binary representation remains unchanged.

(b) Convert −23

First convert +23 into binary.

23_{10} = **00010111_2**

Take the 2's complement to represent −23.

1's Complement:

00010111

↓

11101000

Add 1:

11101000

+00000001

11101001

Hence,

−23 = 11101001$_2$

2. Sign Bit Analysis

The Most Significant Bit (MSB) represents the sign.

Number	Binary	Sign Bit	Meaning
+45	00101101	0	Positive
−23	11101001	1	Negative

Since both numbers are already 8 bits long, no additional sign extension is required.

3. Addition Operation (+45 + (−23))

The processor adds both binary numbers.

```
  00101101  (+45)
+ 11101001  (−23)
------------
1 00010110
```

The carry beyond the eighth bit is discarded.

Final Result:

00010110

Converting back to decimal:

$00010110_2 = \mathbf{22}_{10}$

Therefore,

+45 + (−23) = +22

4. Overflow Analysis for Addition

Overflow occurs only when:

- Two positive numbers produce a negative result, or
- Two negative numbers produce a positive result.

In this case:

Positive + Negative = Positive

Since the operands have opposite signs, overflow cannot occur.

Overflow Status: No Overflow

5. Subtraction Operation (+45 − (−23))

The processor performs subtraction by adding the 2's complement of the second operand.

Since the second operand is already −23,

+45 − (−23) = +45 + (+23)

Binary of +23:

00010111

Addition:

```
  00101101  (+45)
+ 00010111  (+23)
------------
  01000100
```

Converting back to decimal:

$01000100_2 = \mathbf{68}_{10}$

Therefore,

+45 − (−23) = +68

6. Overflow Analysis for Subtraction

The operands being added are:

- +45
- +23

The result is +68, which lies within the 8-bit signed range (−128 to +127).

Therefore, **no overflow occurs**.

7. Processor Overflow Detection

A processor detects overflow by examining the carry into and carry out of the sign bit (MSB).

- If the carry into the MSB equals the carry out of the MSB, **no overflow** occurs.
- If they differ, **overflow** has occurred.

The processor also checks operand signs:

- Positive + Positive → Negative → Overflow
- Negative + Negative → Positive → Overflow
- Positive + Negative → Overflow cannot occur

Since neither arithmetic operation satisfies the overflow conditions, the overflow flag remains **0**.

8. Final Interpretation

Operation	Binary Result	Decimal Result	Overflow
+45 + (−23)	00010110	**+22**	No
+45 − (−23)	01000100	**+68**	No

Conclusion

The processor successfully converts the operands into **8-bit 2's complement format**, aligns their sign bits, and performs arithmetic using binary addition. For subtraction, it internally converts the operation into addition by using the 2's complement method. The results are **+22** for addition and **+68** for subtraction. Both values are within the valid 8-bit signed range (−128 to +127), so **no overflow occurs**. The processor confirms this by verifying that the carry into the sign bit matches the carry out and by ensuring that the result's sign is consistent with the operand signs. This demonstrates the correct execution of signed arithmetic using 2's complement representation.

2. A high-performance computing system replaces a ripple carry adder with a 4-bit carry look-ahead adder (CLA) to reduce computation delay. Given two binary inputs, analyze how generate and propagate signals are formed and how carries are computed in advance. Compare the time delay of CLA with ripple carry addition and justify, with reasoning, why CLA improves speed in modern processors.

Given:

A high-performance computing system replaces a **4-bit Ripple Carry Adder (RCA)** with a **4-bit Carry Look-Ahead Adder (CLA)** to reduce computation delay.

Consider two 4-bit binary numbers:

- **A = 1011**
- **B = 0110**
- Initial Carry (C_0) = 0

The objective is to analyze how the CLA generates **Generate (G)** and **Propagate (P)** signals, computes carries in advance, and explain why it is faster than a Ripple Carry Adder.

Step 1: Input Bits

Bit Position	A	B
A_3 B_3	1	0
A_2 B_2	0	1
A_1 B_1	1	1
A_0 B_0	1	0

Step 2: Generate (G) and Propagate (P) Signals

For each bit,

Generate:

$$G_i = A_i \times B_i$$

A carry is generated when both input bits are 1.

Propagate:

$$P_i = A_i \oplus B_i$$

A carry is propagated when exactly one input bit is 1.

Calculate G and P

Bit	A	B	G = A·B	P = A ⊕ B
0	1	0	0	1
1	1	1	1	0
2	0	1	0	1
3	1	0	0	1

Therefore,

- $G_0 = 0$
- $G_1 = 1$
- $G_2 = 0$
- $G_3 = 0$
- $P_0 = 1$
- $P_1 = 0$
- $P_2 = 1$
- $P_3 = 1$

Step 3: Carry Computation in Advance

Unlike Ripple Carry Adder, CLA computes all carries simultaneously.

Carry equations:

$\mathbf{C_1 = G_0 + P_0C_0}$

$= 0 + (1 \times 0)$

$= \mathbf{0}$

$\mathbf{C_2 = G_1 + P_1G_0 + P_1P_0C_0}$

$= 1 + (0 \times 0) + (0 \times 1 \times 0)$

$= \mathbf{1}$

$\mathbf{C_3 = G_2 + P_2G_1 + P_2P_1G_0 + P_2P_1P_0C_0}$

= 0 + (1×1) + 0 + 0

= **1**

$\mathbf{C_4 = G_3 + P_3G_2 + P_3P_2G_1 + P_3P_2P_1G_0 + P_3P_2P_1P_0C_0}$

= 0 + 0 + (1×1×1) + 0 + 0

= **1**

Hence,

Carry	**Value**
C_0	0
C_1	0
C_2	1
C_3	1
C_4	1

Step 4: Sum Calculation

Sum equation:

$$S_i = P_i \oplus C_i$$

Calculate sums

$\mathbf{S_0 = P_0 \oplus C_0}$

$= 1 \oplus 0$

= **1**

$\mathbf{S_1 = P_1 \oplus C_1}$

$= 0 \oplus 0$

= **0**

$\mathbf{S_2 = P_2 \oplus C_2}$

$= 1 \oplus 1$

= **0**

$\mathbf{S_3 = P_3 \oplus C_3}$

$= 1 \oplus 1$

$= \mathbf{0}$

Final Result

Carry = 1

Sum = 0001

Hence,

$\mathbf{1011 + 0110 = 10001_2\ (17_{10})}$

Step 5: Comparison with Ripple Carry Adder

Ripple Carry Adder (RCA)

- Each full adder waits for the carry from the previous stage.
- Carry propagates sequentially from LSB to MSB.
- Total delay increases with the number of bits.

For a 4-bit RCA:

Bit0 → Bit1 → Bit2 → Bit3

If one full adder requires **t** nanoseconds,

Total delay:

$$T_{RCA} = 4t$$

Carry Look-Ahead Adder (CLA)

- Generate and Propagate signals are computed simultaneously.
- Carry outputs are obtained using logic equations instead of waiting for previous carries.
- All carries are available almost at the same time.

Delay:

$$T_{CLA} = t_{GP} + t_{Logic} + t_{Sum}$$

which is approximately **2t**, much less than the ripple carry delay.

Step 6: Why CLA Improves Speed

The Ripple Carry Adder experiences cumulative delay because every stage depends on the previous carry output. As the number of bits increases, the propagation delay also increases linearly, making it slower for large arithmetic operations.

The Carry Look-Ahead Adder overcomes this limitation by generating **Generate (G)** and **Propagate (P)** signals for every bit and computing all carry values in parallel using combinational logic. Since carries are determined in advance rather than propagated sequentially, the overall computation time is significantly reduced.

Modern processors use CLA and related fast-adder architectures because they:

- Reduce carry propagation delay.
- Perform high-speed arithmetic operations.
- Improve processor performance in ALUs.
- Increase instruction execution speed.
- Enhance the efficiency of scientific, graphics, and real-time computing applications.

Conclusion

The **4-bit Carry Look-Ahead Adder (CLA)** computes **Generate (G)** and **Propagate (P)** signals for each bit and determines all carry outputs simultaneously using Boolean logic. For the given inputs (**1011** and **0110**), the carries are computed in advance, producing the final result $\mathbf{10001}_2$ **(**$\mathbf{17}_{10}$**)**. Compared to the Ripple Carry Adder, which waits for each carry to propagate sequentially, the CLA significantly reduces computation delay. This parallel carry computation makes CLA much faster and is the reason it is widely used in modern processors and high-performance computing systems.

3. In a signed multiplication operation, a processor uses Booth's algorithm to multiply two numbers, −6 and +5. Analyze the algorithm step-by-step, including the role of the accumulator, multiplier, and Booth bit, along with arithmetic shifts. Explain how the algorithm efficiently handles signed numbers and reduces the number of addition/subtraction operations compared to traditional multiplication.

Given:

- Multiplicand (**M**) = **−6**
- Multiplier (**Q**) = **+5**

Using **Booth's Algorithm** for signed multiplication.

Step 1: Initialization

- **Accumulator (A)** = 0000
- **Multiplier (Q)** = 0101
- **Booth Bit (Q_{-1})** = 0

Step 2: Booth's Rules

- **Q_0Q_{-1} = 01** → Add **M** to A.
- **Q_0Q_{-1} = 10** → Subtract **M** from A.
- **Q_0Q_{-1} = 00 or 11** → No operation.
- Perform an **arithmetic right shift** after each operation.

Step 3: Multiplication Process

The processor repeatedly checks **Q_0Q_{-1}**, performs the required addition or subtraction, and then executes an arithmetic right shift until all bits are processed.

Step 4: Final Result

The final binary result represents:

−6 × +5 = −30

How Booth's Algorithm is Efficient

- Handles **positive and negative numbers** directly using **2's complement** representation.
- Reduces the number of **addition and subtraction operations** by grouping consecutive 1s in the multiplier.
- Uses **arithmetic right shifts** to preserve the sign bit during multiplication.
- Faster and more efficient than traditional shift-and-add multiplication, making it suitable for modern processors.

Conclusion

Booth's Algorithm multiplies **−6 and +5** by using an **accumulator (A)**, **multiplier (Q)**, and **Booth bit (Q_{-1})**. It performs addition or subtraction based on the Booth encoding, followed by arithmetic shifts. The final result is **−30**, and the algorithm improves performance by minimizing arithmetic operations while efficiently handling signed numbers.

4. A processor is required to perform division of two binary numbers (e.g., 13 ÷ 3) using both restoring and non-restoring division algorithms. Analyze both methods step-by-step, showing intermediate remainders and quotient formation. Compare the two approaches in terms of number of steps, complexity, and efficiency, and explain why non-restoring division is often preferred in modern systems.

Given:

- Dividend = **$13 = 1101_2$**

- Divisor = **3 = 0011_2**

The processor performs division using **Restoring** and **Non-Restoring Division** algorithms.

1. Restoring Division

- Subtract the divisor from the remainder.
- If the remainder is **positive**, write **1** in the quotient.
- If the remainder is **negative**, **restore** the previous remainder by adding the divisor back and write **0** in the quotient.
- Repeat until all bits are processed.

Result:

- **Quotient = 0100_2 (4)**
- **Remainder = 0001_2 (1)**

2. Non-Restoring Division

- Subtract the divisor if the remainder is positive; add the divisor if the remainder is negative.
- No restoring step is performed.
- The quotient is updated after each iteration.
- If the final remainder is negative, add the divisor once to correct it.

Result:

- **Quotient = 0100_2 (4)**
- **Remainder = 0001_2 (1)**

Comparison

Restoring Division	**Non-Restoring Division**
Restores negative remainder	No restoring operation
More arithmetic operations	Fewer arithmetic operations
Higher execution time	Faster execution
More complex control	Simpler and efficient

Conclusion

Both algorithms produce the same result: **13 ÷ 3 = Quotient 4, Remainder 1**. However, **Non-Restoring Division** is preferred in modern processors because it eliminates the restoring step, reducing the number of operations and improving execution speed and overall efficiency.

5. A scientific application requires precise handling of real numbers using the IEEE 754 standard for floating-point representation. Given a decimal number (e.g., −13.25), analyze how it is represented in both single precision and double precision formats, including sign, exponent, and mantissa fields. Further, explain how floating-point addition is performed between two such numbers, highlighting issues like normalization, rounding, and precision loss.

Given:

Decimal number = **−13.25**

Binary equivalent:

$-13.25 = -1101.01_2 = -1.10101 \times 2^3$

1. IEEE 754 Single Precision (32-bit)

- **Sign bit (S)** = **1** (negative)
- **Exponent** = **3 + 127 = 130 = 10000010_2**
- **Mantissa** = **10101000000000000000000**

Representation:

1 | 10000010 | 10101000000000000000000

2. IEEE 754 Double Precision (64-bit)

- **Sign bit (S)** = **1**
- **Exponent** = **3 + 1023 = 1026 = 10000000010_2**
- **Mantissa** = **1010100000000000000000000000000000000000000000000000**

Representation:

1 | 10000000010 | 1010100000000000000000000000000000000000000000000000

3. Floating-Point Addition

To add two floating-point numbers:

1. Compare and align the exponents.
2. Shift the mantissa of the smaller exponent.
3. Add or subtract the mantissas.
4. Normalize the result.
5. Round the mantissa if required.
6. Store the final sign, exponent, and mantissa.

Issues in Floating-Point Addition

- **Normalization:** Adjusts the result to the standard IEEE 754 format.
- **Rounding:** Applied when extra bits cannot be stored.
- **Precision Loss:** Small errors may occur due to the limited mantissa size, especially when adding numbers with very different magnitudes.

Conclusion

IEEE 754 represents **−13.25** using **Sign, Exponent, and Mantissa** fields in both **32-bit (single precision)** and **64-bit (double precision)** formats. During floating-point addition, exponents are aligned, mantissas are added, and the result is normalized and rounded. Although IEEE 754 provides efficient real-number representation, **rounding and finite precision can introduce small numerical errors**.

Code of Conduct:

I (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

MARKS ALLOCATION

	Problem Understanding (20%)	Method Selection (20%)	Solution Process (25%)	Accuracy of Calculation (20%)	Interpretation of Results (15%)
Q1					
Q2					
Q3					
Q4					
Q5					

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	20%	Clearly interprets problem, identifies all parameters and requirements accurately	Understands problem with minor gaps	Partial understanding; misses key elements	Misinterprets or unable to understand problem
Method Selection	20%	Selects most appropriate method/formula with strong justification	Selects correct method with minor errors	Inappropriate or partially correct method	Unable to select method
Solution Process	25%	Logical, systematic steps with correct approach throughout	Mostly correct steps with minor mistakes	Disorganized steps; multiple errors	No clear process
Accuracy of Calculation	20%	All calculations accurate; correct final answer	Minor calculation errors	Frequent errors; incorrect final answer	No valid calculations
Interpretation of Results	15%	Clearly interprets results with units and engineering relevance	Basic interpretation with minor omissions	Limited or unclear interpretation	No interpretation